Technical Module Description: `testlibgcs.c`

1. Overview

The `testlibgcs.c` module serves as a comprehensive **test harness and verification utility** for the `libgcs` (GCSORT) library. It is designed to validate the high-performance sorting, merging, and record transformation capabilities of the library. The module establishes a fully cross-platform execution environment that interfaces directly with COBOL sequential file formats via the GnuCOBOL runtime environment (`libcob`).

2. Key Functional Components

2.1 Command-Line Interface & Parameter Parsing

The module utilizes a centralized structure (`CmdLineParams`) to capture runtime configurations.

- **Cross-Platform Compatibility:** Features a built-in, minimal implementation of `getopt_long` for environments lacking native POSIX support (such as Microsoft Visual C++ / MSVC).
- **Parameter Validation:** Ensures mutually exclusive constraints are respected; for instance, preventing simultaneous execution of `INREC` / `OUTREC` , `INCLUDE_` / `OMIT` routines.
- **Dynamic Fallbacks:** Programmatically allocates memory and applies default file paths if parameters are omitted at execution.

2.2 GCSORT Engine Initialization & Configuration

The module configures the core operational behaviors of the `libgcs` library prior to execution:

- **Collating Sequences:** Supports Native, ASCII, and EBCDIC character set sequence evaluation.
- **Sign Representation:** Configures numeric sign conventions to align with either standard ASCII or Mainframe-style EBCDIC formats.
- **Multi-Key Sorting Definiton:** Establishes structured composite sort keys using predefined positions, lengths, data types (e.g., Zoned Decimal `ZD`), and direction vectors (Ascending/Descending).

2.3 Record Formatting & Data Refactoring (INREC/OUTREC)

Demonstrates advanced record-level structural refactoring both before sorting (`INREC`) and prior to final output writing (`OUTREC`):

- **Data Injection:** Injects literal alphanumeric constants at precise byte offsets.
- **Field Mapping:** Re-maps explicit byte ranges from input offsets onto new destination positions.
- **Data Padding:** Automates buffer length normalizations using customized hex or character padding sequences.

2.4 Record Selection & Conditional Filtering (INCLUDE/OMIT)

Demonstrates advanced record-level filtering criteria applied during the file reading phase to selectively isolate or skip data blocks:

- **Conditional Comparisons:** Evaluates input data using explicit field-to-field comparisons or field-to-constant value matching.
- **Logical Composition:** Combines multiple search criteria into advanced evaluation trees using boolean `AND` / `OR` logical operators.
- **Exclusive Routing:** Restricts processing workloads by either explicitly capturing matching rows (INCLUDE) or skipping specified entries (OMIT) in a mutually exclusive context.

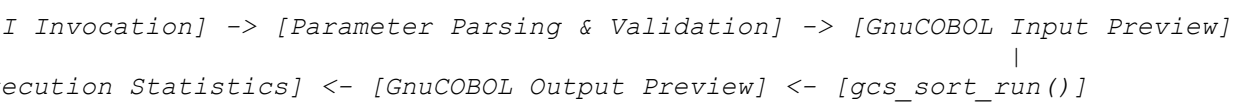
2.4 GnuCOBOL Runtime File Interoperability

Rather than employing standard C stream I/O (`fopen`/`fread`), the module implements low-level bindings with the GnuCOBOL (`libcob`) file-handling subsystem:

- **Structure Instantiation:** Manually allocates and populates native `cob_file` and `cob_field` control blocks.
- **Sequential I/O Control:** Operates on variable-length sequential organization files (`COB_ORG_SEQUENTIAL`), continuously evaluating standard 2-byte COBOL File Status codes to monitor operations and detect End-Of-File (EOF) conditions.

---

3. Program Flow and Execution Architecture



1. **Initialization:** The program registers command-line inputs and verifies execution safety rules.

2. **File Validation:** The input file is opened via `cob_open`, its initial layout is verified, a subset of sample records is printed to stdout, and the file descriptor is safely closed.
3. **Execution:** The GCSORT environment is initialized, sort keys and file characteristics are bound to the engine, record filters are applied, and `gcs_sort_run()` executes the sorting or merging algorithm.
4. **Finalization & Diagnostics:** The generated output file is read back through GnuCOBOL descriptors for visual verification. Comprehensive processing metrics (such as total record counts, written rows, and discarded items) are generated before memory cleanup routines take place.
